FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

VOZES DIGITAIS: DA FALA AO TEXTO E DO TEXTO À FALA

3

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Representação do reconhecimento de fala.....6
Figura 2 - Processo de reconhecimento de fala.....7
Figura 3 - O preenchimento automático é uma aplicação clássica de um modelo de linguagem.....8
Figura 4 - Exemplos de assistentes virtuais, dependentes de áudio para funcionamento.....9
Figura 5 - Cena do filme “Gravidade”.....19
Figura 6 - Componentes de uma onda sonora.....20
Figura 7 - Frequências de um som.....20
Figura 8 - Amplitude e período de uma onda sonora.....21
Figura 9 - Exemplo de um espectro, relacionando amplitude com frequências.....22
Figura 10 - Exemplo de um espectrograma.....23
Figura 11 - Aplicações de data augmentation no domínio do espectrograma.....24
Figura 12 - Exemplos de fonemas.....25
Figura 13 - Etapas do reconhecimento da voz.....26
Figura 14 - Aplicações de Data Augmentation no domínio do áudio.....31
Figura 15 - Aplicações de Data Augmentation no domínio do espectrograma.....32
Figura 16 - Saída do Código-fonte 2.....35
Figura 17 - Saída do Código-fonte 3.....35
Figura 18 - Saída do Código-fonte 4.....36
Figura 19 - Saída do Código-fonte 5.....36
Figura 20 - Saída do Código-fonte 6.....37
Figura 21 - Saída do Código-fonte 7.....38
Figura 22 - Saída do Código-fonte 8.....39
Figura 23 - Saída do Código-fonte 9.....40
Figura 24 - Saída do Código-fonte 10.....41
Figura 25 - Saída do Código-fonte 11.....42
Figura 26 - Saída do Código-fonte 12.....42
Figura 27 - Saída do Código-fonte 18.....47
Figura 28 - Saída do Código-fonte 19.....47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Comparação de features de dados complexos 29

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Imports e carregamento do arquivo de áudio 34
Código-fonte 2 - Exibindo o *sample rate* padrão da biblioteca Librosa 35
Código-fonte 3 - Exibindo as amplitudes do áudio 35
Código-fonte 4 - Exibindo o próprio arquivo de áudio 35
Código-fonte 5 - Exibindo o espectrograma padrão da biblioteca Librosa 36
Código-fonte 6 - Exibindo a magnitude do espectrograma padrão da biblioteca Librosa 37
Código-fonte 7 - Convertendo o espectrograma padrão para a escala decibéis.....38
Código-fonte 8 - Exibindo o espectrograma em escala decibéis para processamento de CNNs.....39
Código-fonte 9 - Utilização padrão do gTTS 40
Código-fonte 10 - Línguas suportadas pelo gTTS.....41
Código-fonte 11 - Gerando áudios diversos no gTTS 41
Código-fonte 12 - Instalação das bibliotecas.....42
Código-fonte 13 - Solicitando a captura de áudio pelo microfone 45
Código-fonte 14 - Testando a captura do áudio 45
Código-fonte 15 - Realizando os imports necessários 46
Código-fonte 16 - Salva num .wav o áudio capturado.....46
Código-fonte 17 - Transcrição de áudio em português 46
Código-fonte 18 - Exemplo de transcrição de áudio em português.....46
Código-fonte 19 - Exemplo de transcrição em loop de áudios em português 47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 VOZES DIGITAIS: DA FALA AO TEXTO E DO TEXTO À FALA 6
1.1 Definições e contextualizações 6
1.2 Aplicações 9
1.2.1 Aplicações de sintetização 10
1.2.1.1 Músicas 10
1.2.1.2 Educação 11
1.2.1.3 Autenticação & Segurança 11
1.2.2 Aplicações de reconhecimento 12
1.2.2.1 Acessibilidade e inclusão 12
1.2.2.2 Atendimento ao cliente 12
1.2.2.3 Transcrição automática 13
1.3 Desafios 13
1.3.1 Desafios de sintetização 13
1.3.2 Desafios de reconhecimento 15

2 FUNDAMENTOS DO ÁUDIO E DA FALA 18
2.1 Natureza do áudio 18
2.2 Componentes do áudio 19
2.3 Espectro e Espectrogramas 22
2.4 Princípios da fala 24
2.5 Etapas do reconhecimento da fala 26

3 MODELOS DE DEEP LEARNING PARA ÁUDIO 29
3.1 Features tradicionais e modernas 29
3.2 Aumentação de dados 29
3.2.1 Técnicas de Data Augmentation no domínio do áudio 30
3.2.2 Técnicas de Data Augmentation no domínio do espectrograma 31
3.3 Modelos profundos mais utilizados 32

4 HANDS ON: PRATICANDO STT E TTS 34
4.1 Gerando espectrogramas 34
4.2 TTS 40
4.3 SST 42

CONCLUSÃO 48

REFERÊNCIAS 49

GLOSSÁRIO 51

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 VOZES DIGITAIS: DA FALA AO TEXTO E DO TEXTO À FALA

1.1 Definições e contextualizações

O reconhecimento de fala é uma tecnologia que permite que computadores, dispositivos e sistemas interpretem e compreendam a linguagem falada por seres humanos. Em termos mais técnicos, trata-se do processo de converter palavras faladas em texto digital utilizando algoritmos avançados de Processamento de Sinais e Aprendizado de Máquina. Esse processo envolve várias etapas complexas, como captura de áudio, segmentação de fala, extração de características, e finalmente a conversão do som em texto legível por meio de Modelos Estatísticos ou Redes Neurais.

Figura 1 - Representação do reconhecimento de fala
Fonte: Inforchannel (2023)

O reconhecimento de fala pode ser visto como uma **interface entre seres humanos e máquinas**, onde a linguagem falada — uma das formas mais naturais e intuitivas de comunicação — é convertida em comandos ou textos para que uma máquina consiga entender e responder. Este campo de estudo, que já existe há várias décadas, evoluiu consideravelmente graças aos avanços em Inteligência Artificial, Redes Neurais Profundas e Big Data, permitindo que sistemas modernos alcancem níveis de precisão e usabilidade antes inimagináveis.

Um dos principais componentes do reconhecimento de fala é o **Processamento de Sinais de Áudio**. Quando uma pessoa fala, o som produzido é

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

capturado por um microfone e convertido em um sinal elétrico ou digital. Esse sinal precisa ser processado para remover ruídos, normalizar volumes e preparar a fala para a análise subsequente. O sinal de áudio, que inicialmente é um fluxo contínuo de dados, é então segmentado em unidades menores chamadas **frames**, que representam curtos intervalos de tempo durante os quais o áudio é analisado para identificar características fundamentais da fala, como frequência, tom e amplitude.

Essas características são utilizadas para identificar **fonemas**, que são os menores componentes sonoros da fala humana. Os fonemas são combinados para formar palavras, e a partir daí, frases. No entanto, essa tarefa é incrivelmente desafiadora devido à complexidade da linguagem humana, como veremos adiante.

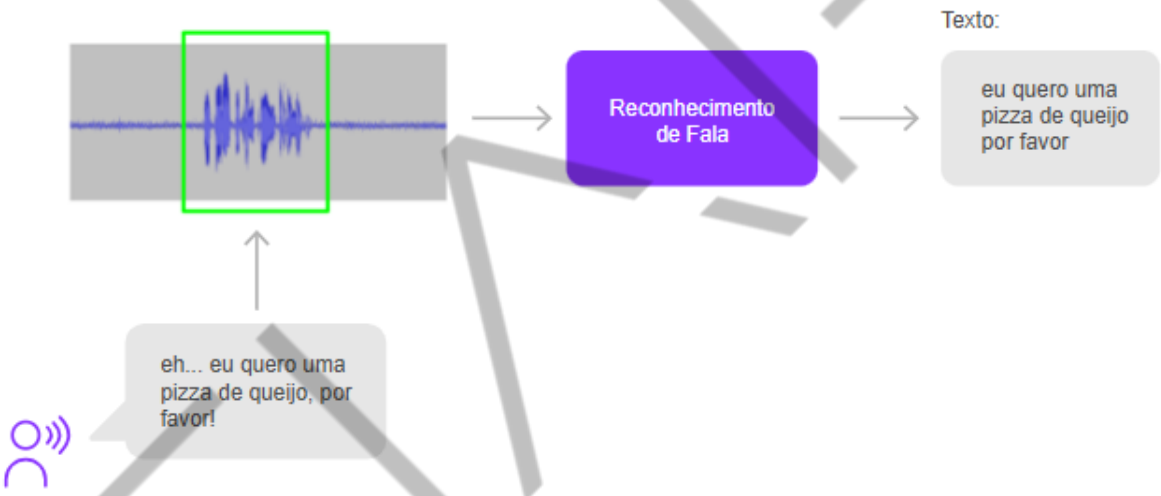

Figura 2 - Processo de reconhecimento de fala
Fonte: CPQD (2017), adaptado por FIAP (2025)

Os modelos de reconhecimento de fala modernos frequentemente empregam **Redes Neurais Profundas (DNNs)**, que são treinadas com enormes quantidades de dados de fala para reconhecer padrões e melhorar a precisão na identificação de palavras. Essas Redes Neurais são capazes de aprender nuances na pronúncia e fazer previsões baseadas em contextos anteriores. Isso significa que, ao ouvir uma palavra ou frase, o sistema não apenas compara sons individuais com uma base de dados, mas também leva em consideração o contexto mais amplo da conversação para melhorar a precisão da interpretação.

Outro aspecto crucial no reconhecimento de fala é o uso de **Modelos de Linguagem (Language Models)**, que são usados para prever a sequência mais

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

provável de palavras com base nas palavras já reconhecidas. Esses modelos podem ser baseados em n-gramas, onde a probabilidade de uma palavra aparecer é calculada com base em uma sequência de n palavras anteriores, ou em modelos mais avançados como LSTM e Transformers, que podem capturar dependências de longo alcance em uma frase ou parágrafo. Observe a figura:

Figura 3 - O preenchimento automático é uma aplicação clássica de um modelo de linguagem
Fonte: Parse.ly (2025)

A importância do reconhecimento de fala nas aplicações modernas não pode ser subestimada. À medida que a tecnologia se torna cada vez mais integrada ao cotidiano das pessoas, a capacidade de interagir com máquinas de forma natural e intuitiva através da fala abre novas possibilidades para inovação e acessibilidade. Isso inclui, mas não se limita apenas a assistentes pessoais inteligentes, sistemas de transcrição automática, e interfaces de usuário baseadas em voz.

Além disso, o reconhecimento de fala também é fundamental para a acessibilidade, permitindo que pessoas com deficiências físicas ou visuais interajam com tecnologias de uma maneira que antes não era viável. Ao eliminar a necessidade de dispositivos de entrada tradicionais, como teclados e mouses, o reconhecimento de fala democratiza o acesso à tecnologia, tornando-o mais inclusivo e universal.

Assim como o reconhecimento, a sintetização de voz também ocupa um papel importante na Inteligência Artificial. Conhecida como Text-to-Speech (TTS), é a tecnologia que transforma texto escrito em fala humana artificial. Ela permite que máquinas “falem”, criando interações mais naturais entre humanos e sistemas computacionais, com aplicações que vão desde assistentes virtuais até acessibilidade para pessoas com deficiência visual. Os sistemas modernos de TTS utilizam Redes Neurais Profundas, que conseguem gerar vozes com entonação, ritmo e pausas realistas, muitas vezes indistinguíveis da fala humana. Essa evolução tem ampliado o

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

impacto da sintetização de voz em educação, atendimento automatizado, leitura de conteúdo e inclusão digital.

Com a contínua melhoria dos sistemas de reconhecimento e sintetização de fala, a capacidade de usar a voz como um meio natural e eficiente de controlar e interagir com dispositivos continuará a crescer, tornando-se uma peça central no futuro da computação.

1.2 Aplicações

No setor de serviços, assistentes virtuais como Siri, Alexa e Google Assistant dependem fortemente do processamento de áudio para compreender e responder a comandos de voz. Esses sistemas melhoram a eficiência e a conveniência do serviço ao cliente, permitindo interações rápidas e personalizadas. Além disso, no entretenimento, o processamento de áudio é essencial para a criação de trilhas sonoras, dublagem de filmes, produção musical e desenvolvimento de jogos, proporcionando uma experiência mais rica e envolvente para o usuário.

O processamento de áudio é essencial para acessibilidade. Reconhecimento de voz e síntese de fala auxiliam pessoas com deficiência a usar dispositivos de forma autônoma. Enquanto a transcrição automática, que converte áudio em texto, beneficia aqueles com deficiência auditiva.

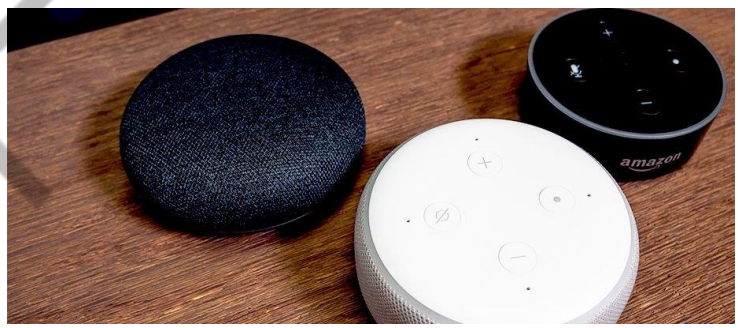

Figura 4 - Exemplos de assistentes virtuais, dependentes de áudio para funcionamento
Fonte: AudioPrime (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.2.1 Aplicações de sintetização

1.2.1.1 Músicas

Na indústria musical, temos diversas aplicações que vão desde a classificação de gênero musical até a geração automática de músicas e transcrição de letras. Esses avanços tecnológicos não só aprimoram a experiência do usuário, mas também revolucionam a forma como a música é criada, distribuída e consumida.

A **classificação de gênero musical** é uma aplicação fundamental do processamento de áudio. Utilizando técnicas de Aprendizado de Máquina, é possível analisar características específicas das faixas de áudio, como ritmo, melodia, harmonia e timbre, para categorizar músicas em gêneros distintos. Por exemplo, algoritmos são utilizados para extrair características de áudio que ajudam a diferenciar entre gêneros como rock, jazz, clássico e eletrônico. Plataformas de streaming como Spotify e Apple Music usam essas tecnologias para organizar vastas bibliotecas musicais e melhorar a navegabilidade dos usuários.

Além disso, os **sistemas de recomendação de músicas** são outra aplicação importante do processamento de áudio. Esses sistemas analisam as preferências e o histórico de escuta dos usuários para sugerir novas músicas que eles provavelmente gostarão. Em relação às técnicas de Filtragem Colaborativa, Análise de Conteúdo e Aprendizado Profundo, elas são combinadas para entender as preferências musicais e comportamentos dos ouvintes. Por exemplo, o algoritmo de recomendação do Spotify utiliza uma combinação de análise de áudio e dados de usuário para criar playlists personalizadas, o que se tornou um recurso popular para descobrir novas músicas.

A **geração automática de músicas** é uma área emergente que utiliza Redes Neurais e algoritmos generativos para criar composições musicais originais. Projetos como o Magenta, desenvolvido pelo Google, exploram como o Aprendizado de Máquina pode ser usado para compor músicas, oferecendo ferramentas para músicos e compositores criarem peças a partir de sugestões algorítmicas. Essas tecnologias não só auxiliam na criação musical, mas também podem gerar música de fundo personalizada para jogos, filmes e outras mídias.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.2.1.2 Educação

Na educação, o processamento de áudio está sendo utilizado para criar ferramentas que tornam o aprendizado mais acessível e eficiente. Tecnologias de reconhecimento de fala, por exemplo, permitem a **transcrição automática de aulas e palestras**, facilitando o acesso de conteúdos educativos para estudantes com dificuldades auditivas ou para aqueles que preferem revisar o material posteriormente.

Além disso, **assistentes virtuais educacionais**, equipados com capacidades de processamento de áudio, podem interagir com os alunos, responder a perguntas e fornecer feedback imediato, tornando o aprendizado mais dinâmico e personalizado.

Outra aplicação importante do processamento de áudio na educação é o desenvolvimento de plataformas de e-learning que utilizam análise de voz para **avaliar a pronúncia e a fluência dos estudantes em diferentes idiomas**. Ferramentas como Duolingo e Rosetta Stone empregam algoritmos avançados para analisar a fala dos usuários e oferecer correções em tempo real, ajudando-os a melhorar suas habilidades linguísticas de maneira interativa e envolvente.

1.2.1.3 Autenticação & Segurança

No campo da segurança dos usuários, o processamento de áudio desempenha um papel crucial na detecção e prevenção de incidentes perigosos. **Sistemas de vigilância** baseados em áudio monitoram ambientes em tempo real, identificando sons anômalos, como vidros quebrando, gritos e disparos de armas de fogo, e alertam as autoridades imediatamente. Esses sistemas são particularmente úteis em locais públicos, como escolas, aeroportos e estações de transporte, onde a rápida identificação de ameaças pode salvar vidas e prevenir danos.

Além disso, o processamento de áudio é utilizado em dispositivos de **segurança doméstica**, como campainhas inteligentes e sistemas de alarme, que podem reconhecer comandos de voz e responder a eventos sonoros específicos. Por exemplo, um alarme de segurança pode ser configurado para acionar quando detectar sons de invasão, como arrombamento de portas ou janelas. Esses dispositivos não

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

só aumentam a segurança dos lares, mas também proporcionam tranquilidade aos moradores, permitindo que monitorem suas propriedades remotamente.

A **autenticação por voz** é outra aplicação significativa do processamento de áudio. Bancos e instituições financeiras estão adotando a biometria de voz para verificar a identidade dos clientes durante transações telefônicas, aumentando a segurança e reduzindo o risco de fraude. A autenticação por voz é baseada nas características únicas da fala de cada indivíduo, tornando-a uma forma segura e conveniente de proteção de dados e acesso a informações sensíveis.

1.2.2 Aplicações de reconhecimento

1.2.2.1 Acessibilidade e inclusão

O reconhecimento de fala desempenha um papel fundamental na melhoria da acessibilidade, permitindo que pessoas com diferentes tipos de deficiências interajam com a tecnologia de forma mais fácil e independente. Isso inclui facilitar a comunicação para pessoas com deficiências motoras ou auditivas e fornecer meios alternativos de entrada e controle de dispositivos.

Algumas aplicações famosas neste cenário são o Microsoft Speech Recognition, o Dragon NaturallySpeaking e o Voiceitt. O Dragon NaturallySpeaking é um software de reconhecimento de voz que permite que usuários controlem computadores e criem documentos por meio de comandos de voz, beneficiando especialmente aqueles com mobilidade limitada.

Já o Voiceitt é uma aplicação que ajuda pessoas com fala não padronizada, como aquelas com paralisia cerebral ou derrames, a se comunicarem efetivamente através de tecnologia de reconhecimento de fala adaptativa.

1.2.2.2 Atendimento ao cliente

O reconhecimento de fala é amplamente utilizado em sistemas de atendimento ao cliente para automatizar e agilizar o suporte, permitindo que clientes resolvam

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

problemas comuns ou obtenham informações sem a necessidade de intervenção humana direta. Isso melhora a eficiência operacional e a satisfação do cliente.

1.2.2.3 Transcrição automática

A transcrição automática converte fala em texto escrito, permitindo a documentação rápida e eficiente de discursos, entrevistas, reuniões e outras formas de comunicação verbal. Essa aplicação é particularmente útil para economizar tempo e recursos na criação de registros escritos precisos e facilita o acesso e a análise de informações verbais.

Alguns exemplos de aplicações comerciais para este fim incluem a Otter.ai, a Ver e o Google Live Transcribe.

1.3 Desafios

1.3.1 Desafios de sintetização

O processamento de áudio apresenta uma série de desafios únicos que exigem abordagens especializadas e inovadoras para serem superados. Estes desafios surgem devido à **natureza complexa e variada dos sinais de áudio, assim como às condições em que são capturados e utilizados**. A seguir serão listados alguns exemplos de tais desafios:

- **Variedade e complexidade:** os sinais de áudio podem variar amplamente em termos de duração, frequência, amplitude e conteúdo. Esta variedade torna o processamento de áudio uma tarefa complexa.
    - Por exemplo, um sinal de áudio pode conter fala, música, ruído de fundo, e sons ambientais, cada um com suas próprias características acústicas. Extrair informações significativas de tais sinais heterogêneos requer técnicas avançadas de análise e modelagem.
- **Ruído e interferência:** o ruído e a interferência são problemas comuns no processamento de áudio. Fontes de ruído podem incluir sons ambientes,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

como tráfego ou vento, bem como interferências eletrônicas. O ruído pode mascarar ou distorcer os sinais de interesse, dificultando a análise precisa.

  - Técnicas de filtragem e supressão de ruído são frequentemente necessárias para melhorar a qualidade do áudio antes de qualquer processamento adicional.

- **Variação de volume e entonação**: no caso da fala, os sinais de áudio capturados em diferentes ambientes ou por diferentes dispositivos podem apresentar variações significativas em volume e entonação. Por exemplo, gravações realizadas em ambientes ruidosos podem ter volumes muito baixos, enquanto outras podem ter volumes muito altos. A entonação também pode variar significativamente entre falantes ou músicos.

  - Normalizar essas variações é essencial para garantir que os modelos de processamento de áudio possam operar de maneira consistente e eficaz.

- **Recursos computacionais:** o processamento de áudio em tempo real pode ser computacionalmente intensivo. Modelos complexos, como Redes Neurais Profundas, requerem uma quantidade significativa de poder de processamento e memória. Isso pode ser um desafio, especialmente em dispositivos com recursos limitados, como smartphones e dispositivos de IoT.

  - A otimização de algoritmos e o uso eficiente de recursos computacionais são essenciais para a implementação prática de sistemas de processamento de áudio.

- **Rotulagem e anotação de dados:** para treinar modelos de Machine Learning, é necessário um grande volume de dados rotulados e anotados. No caso do áudio, a rotulagem manual pode ser um processo demorado e caro, pois requer especialistas que possam identificar e marcar eventos específicos dentro do sinal de áudio. Além disso, garantir a consistência e a precisão das anotações pode ser um desafio, especialmente em grandes conjuntos de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Além dos desafios citados, a **privacidade e a segurança dos dados de áudio** também são questões importantes, especialmente quando se trata de gravações de voz que podem conter informações pessoais sensíveis. Proteger esses dados contra acesso não autorizado e garantir seu uso ético e conforme as regulamentações de privacidade são desafios críticos. Técnicas de anonimização e criptografia podem ser empregadas para mitigar esses riscos.

1.3.2 Desafios de reconhecimento

A implementação bem-sucedida de sistemas de reconhecimento de fala é acompanhada por uma série de desafios complexos. Estes desafios surgem principalmente devido à natureza intrincada e variável da comunicação humana, que envolve sotaques, entonações, ruídos de fundo e ambiguidade linguística. Exemplos de tais desafios são:

- **Ambiguidade da fala:** a ambiguidade da fala refere-se à capacidade de certas palavras ou frases terem múltiplos significados, dependendo do contexto. Na linguagem falada, uma única sequência de sons pode corresponder a várias palavras ou frases diferentes, o que pode levar a mal-entendidos se o sistema de reconhecimento de fala não for capaz de interpretar corretamente o contexto.
    - **Exemplo:** considere a frase: “A porta está aberta”. Dependendo do contexto, "porta" pode se referir a uma porta de carro, uma porta de casa ou até mesmo a porta de um computador (porta de rede). Se o sistema de reconhecimento de fala não tiver contexto suficiente, pode não identificar corretamente o significado da palavra "porta".
    - **Mitigação:** para mitigar a ambiguidade da fala, os sistemas de reconhecimento de fala modernos utilizam **modelos de linguagem baseados em contexto**. Esses modelos analisam as palavras precedentes e subsequentes para prever a interpretação mais provável de uma palavra ou frase ambígua. Além disso, incorporar **modelos de linguagem mais avançados**, como Redes Neurais Recorrentes (RNNs)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

ou Transformers, pode melhorar significativamente a compreensão do contexto, reduzindo assim a ambiguidade.

- **Sotaques e variações na pronúncia:** as variações regionais e pessoais na pronúncia, conhecidas como sotaques, representam um dos maiores desafios no reconhecimento de fala. Sotaques podem alterar a forma como os sons são produzidos e percebidos, o que pode levar a erros de interpretação por parte do sistema.
  - **Exemplo:** a palavra "porta" pode ser pronunciada de maneira diferente dependendo da região, por exemplo, puxando mais o “R”. Se um sistema de reconhecimento de fala foi treinado predominantemente com exemplos de fala de uma região específica, ele pode ter dificuldade em reconhecer as pronúncias de falantes de outras regiões.
  - **Mitigação:** uma das abordagens para mitigar este desafio é **aumentar a diversidade dos dados de treinamento**. Ao incluir exemplos de fala de pessoas com diferentes sotaques, o sistema de reconhecimento pode aprender a identificar e interpretar as variações na pronúncia.
- **Ruídos e interferência na captura do áudio:** ruídos de fundo, como sons de trânsito ou conversas paralelas, podem interferir na qualidade do áudio capturado, tornando mais difícil para o sistema de reconhecimento de fala identificar e processar a fala corretamente. O ruído de fundo pode distorcer palavras importantes, gerando detecções falsas ou resultando em transcrições imprecisas.
  - **Exemplo:** imagine uma pessoa tentando utilizar um assistente virtual em um ambiente barulhento, como em uma cafeteria, por exemplo. O som ambiente pode interferir na clareza do comando de voz, levando o assistente a não entender ou interpretar incorretamente o comando.
  - **Mitigação:** para lidar com ruídos e interferências, os sistemas de reconhecimento de fala utilizam **técnicas de processamento de sinais**, como **filtros de ruído e algoritmos de cancelamento de eco**, que ajudam a isolar a fala do ruído de fundo. Outra abordagem eficaz é o uso de **redes neurais treinadas para separar a voz do ruído**, permitindo uma interpretação mais precisa em ambientes desafiadores.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Dependências do contexto:** a linguagem falada frequentemente depende do contexto anterior para ser compreendida corretamente. Palavras ou frases isoladas podem ser ambíguas ou incompletas, e sua interpretação correta depende de informações adicionais do que foi dito anteriormente ou do cenário em que a conversa está ocorrendo.
  - **Exemplo:** se alguém disser “Adicione isso à lista”, o comando "isso" é totalmente dependente do que foi mencionado antes. Sem contexto, o sistema de reconhecimento de fala pode não entender ao que "isso" se refere.
  - **Mitigação:** para enfrentar este desafio, os sistemas de reconhecimento de fala podem utilizar **modelos de linguagem contextual** que não apenas reconhecem palavras individuais, mas também integram essas palavras em um contexto maior para determinar seu significado. **Modelos pré-treinados de NLP** que capturam informações contextuais ao longo de uma conversa também podem ser implementados para melhorar a interpretação em tempo real.

Esses desafios destacam a complexidade do reconhecimento de fala e os obstáculos que ainda precisam ser superados para que a tecnologia atinja níveis ainda mais altos de precisão e confiabilidade. No entanto, com os avanços contínuos em Aprendizado de Máquina, Processamento de Sinais e Modelos de Linguagem, o reconhecimento de fala está se tornando cada vez mais sofisticado, aproximando-se do objetivo de entender e interpretar a fala humana com a mesma nuance e precisão que os seres humanos são capazes de alcançar.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 FUNDAMENTOS DO ÁUDIO E DA FALA

2.1 Natureza do áudio

O áudio, em sua essência, é a representação de **variações na pressão do ar ao longo do tempo**. Quando uma fonte de som, como as cordas vocais humanas, um instrumento musical ou um alto-falante, vibra, ela gera flutuações na pressão do ar ao seu redor. Essas flutuações são ondas sonoras que se propagam pelo meio (geralmente ar, mas também podem ser outros meios como água ou sólidos) até atingirem os nossos ouvidos, onde são interpretadas como som.

Para entender melhor, imagine uma corda de violão sendo tocada. Quando a corda vibra, ela empurra e puxa as moléculas de ar próximas, criando regiões de compressão (alta pressão) e rarefação (baixa pressão). Essas variações de pressão se propagam em ondas para fora da fonte de som. O ouvido humano detecta essas variações de pressão, e o cérebro as interpreta como diferentes sons, dependendo da frequência, amplitude e timbre das ondas (veremos mais disso adiante).

O som requer um meio material para se propagar, pois depende da vibração das partículas nesse meio. No vácuo, onde não há partículas de ar ou qualquer outro material, não há meio para as ondas sonoras se propagarem. Portanto, o som não pode viajar pelo vácuo. Essa é a razão pela qual o espaço, que é essencialmente um vácuo, é silencioso, apesar de haver muitas atividades e movimentos astronômicos. Neste contexto, podemos citar o filme “Gravidade” de 2013, pois ele é preciso ao mostrar as cenas de explosão no espaço sem som algum.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 5 - Cena do filme “Gravidade”
Fonte: O Globo (2025)

2.2 Componentes do áudio

O áudio, ou som, é composto por várias características fundamentais que ajudam a definir e diferenciar os diversos tipos de sons que ouvimos. Entre os componentes mais importantes estão a **frequência**, a **amplitude**, o **período** e as **ondas sonoras**. Compreender esses conceitos é essencial para entender como o som funciona e como podemos manipulá-lo.

As **ondas sonoras** são o meio pelo qual o som se propaga. Quando uma fonte sonora, como uma corda de violão ou as cordas vocais humanas, vibra, ela gera ondas de pressão que se deslocam através do ar ou outro meio. Essas ondas são chamadas de ondas longitudinais, porque as partículas do meio vibram na mesma direção em que a onda se propaga. As ondas sonoras têm **cristas** (pontos de alta pressão) e **vales** (pontos de baixa pressão), e a distância entre duas cristas consecutivas é chamada de comprimento de onda. Observe a figura a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

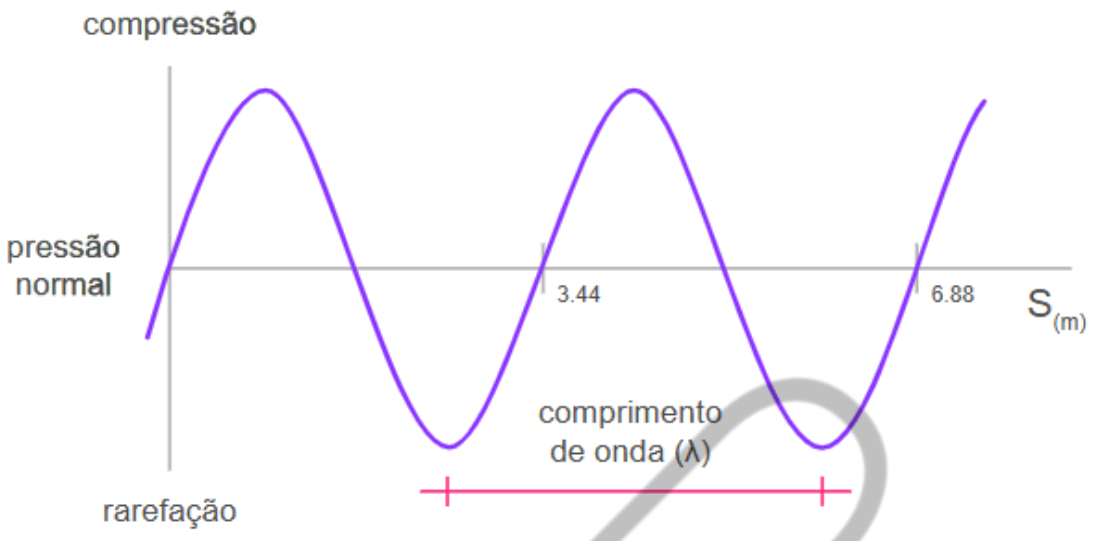

Figura 6 - Componentes de uma onda sonora
Fonte: USP (2025), adaptado por FIAP (2025)

A **frequência** é uma das características mais importantes do som. Ela se refere ao número de ciclos de uma onda sonora que ocorrem em um segundo, medida em Hertz (Hz). Em termos simples, a frequência determina o tom do som: sons com frequências mais baixas são percebidos como graves (por exemplo, o ronco de um motor), enquanto sons com frequências mais altas são percebidos como agudos (como o canto de um pássaro). A frequência está diretamente relacionada ao comprimento da onda sonora: ondas mais curtas têm frequências mais altas, e ondas mais longas têm frequências mais baixas (figura “Frequências de um som”).

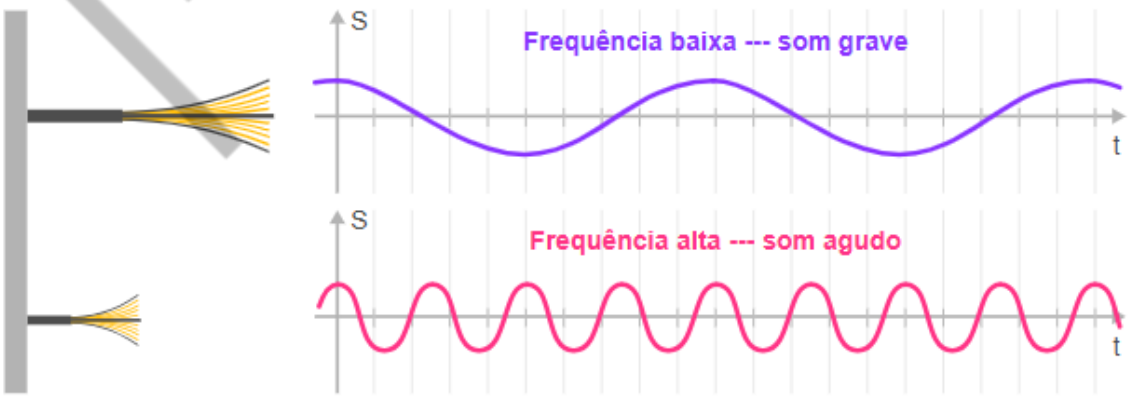

Figura 7 - Frequências de um som
Fonte: UFGRS (2002), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A **amplitude**, por outro lado, refere-se à altura ou intensidade da onda sonora. Ela mede a magnitude das variações de pressão do ar causadas pelo som. Ondas sonoras com maior amplitude são percebidas como sons mais altos, enquanto ondas com menor amplitude são percebidas como sons mais baixos. A amplitude é geralmente representada graficamente pela altura das ondas: ondas mais altas indicam maior amplitude, e, portanto, um som mais alto.

O **período** de uma onda sonora é o tempo que leva para um ciclo completo de uma onda ocorrer. Ele é inversamente relacionado à frequência: se a frequência é alta, o período é curto, e se a frequência é baixa, o período é longo. O período é importante pois ajuda a entender a regularidade e a repetição das ondas sonoras ao longo do tempo. Em termos simples, é o tempo que uma onda leva para repetir seu padrão. Observe a figura a seguir:

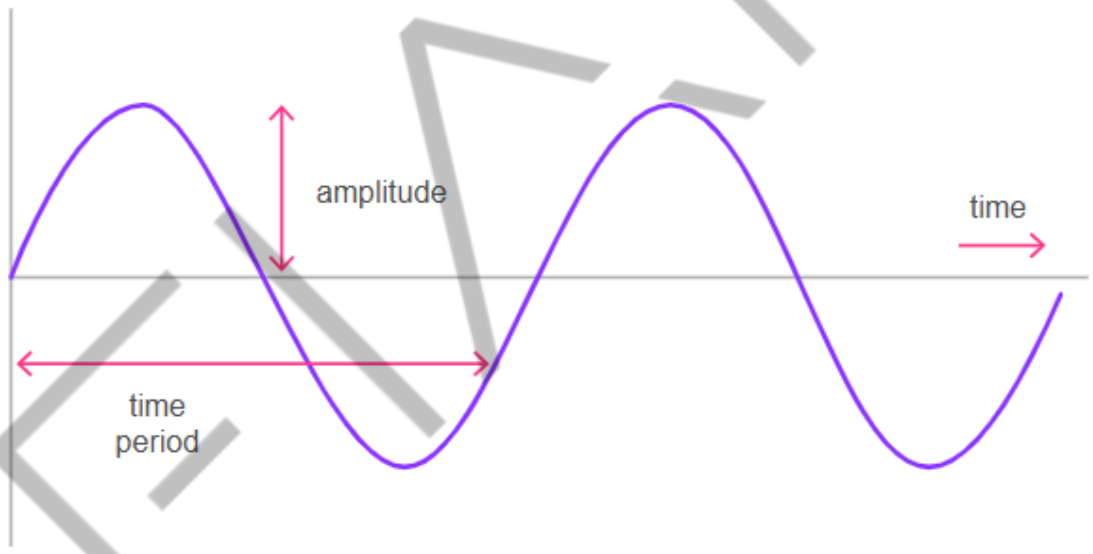

Figura 8 - Amplitude e período de uma onda sonora
Fonte: Vedantu (2025), adaptado por FIAP (2025)

Esses componentes (**frequência, amplitude, período** e **ondas sonoras**) trabalham juntos para definir a natureza do som. Por exemplo, quando ouvimos uma nota musical, sua frequência determina o tom da nota, sua amplitude determina o volume, e o período define a regularidade das ondas sonoras.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Espectro e Espectrogramas

Quando analisamos um som, ele raramente é composto por uma única frequência. Em vez disso, a maioria dos sons que ouvimos no dia a dia, como a fala humana ou a música, são compostos por uma combinação de múltiplas frequências. O **espectro** nos permite visualizar essa combinação, mostrando quais frequências estão presentes e com que intensidade. Essa visualização é essencial para entender a complexidade de um som e para realizar várias tarefas de processamento de áudio, como equalização e filtragem. Assim, o espectro de áudio é uma representação das diferentes frequências presentes em um sinal de áudio e suas respectivas amplitudes.

Para gerar um espectro de áudio, utilizamos uma técnica matemática chamada Transformada de Fourier. **Esta técnica transforma um sinal de áudio do domínio do tempo para o domínio da frequência**. No **domínio do tempo**, o sinal de áudio é uma onda contínua que varia com o tempo. No **domínio da frequência**, o sinal é decomposto em suas componentes de frequência, permitindo-nos ver quais frequências estão presentes e com que amplitude. O resultado dessa transformação é um gráfico onde o eixo horizontal representa as frequências e o eixo vertical representa a amplitude de cada frequência. Observe a figura:

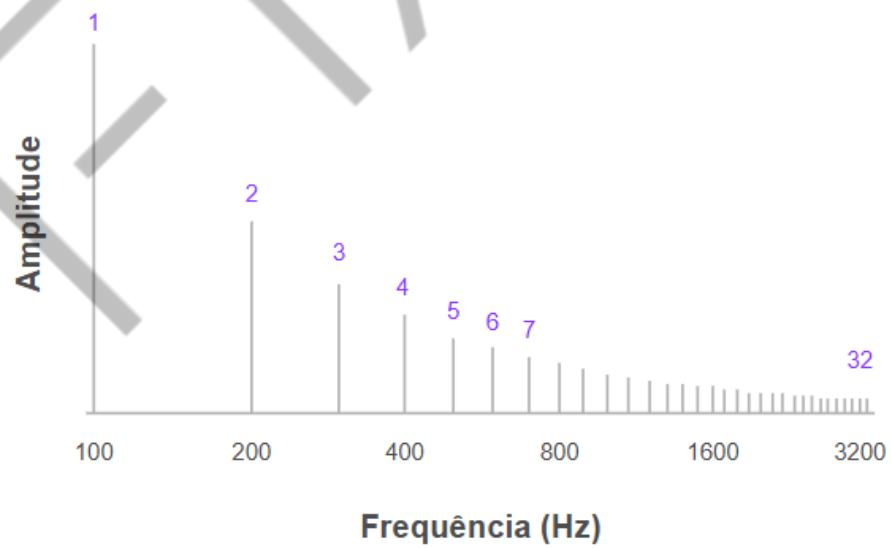

Figura 9 - Exemplo de um espectro, relacionando amplitude com frequências
Fonte: Wikipedia (2006), adaptado por FIAP (2025)

Embora o espectro nos dê uma boa ideia das frequências presentes em um som, ele não mostra como essas frequências mudam ao longo do tempo. Para isso,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

utilizamos **espectrogramas**. Um espectrograma é uma **representação visual que mostra como o espectro de frequências de um sinal varia com o tempo**. Ele é gerado ao aplicar a Transformada de Fourier em janelas temporais sucessivas do sinal de áudio, criando uma série de espectros que são combinados em uma imagem bidimensional. No espectrograma, o eixo horizontal representa o tempo, o eixo vertical representa a frequência, e a intensidade de cada ponto é indicada por uma escala de cores ou níveis de cinza, representando a amplitude. Podemos visualizar melhor na figura “Exemplo de um espectrograma”.

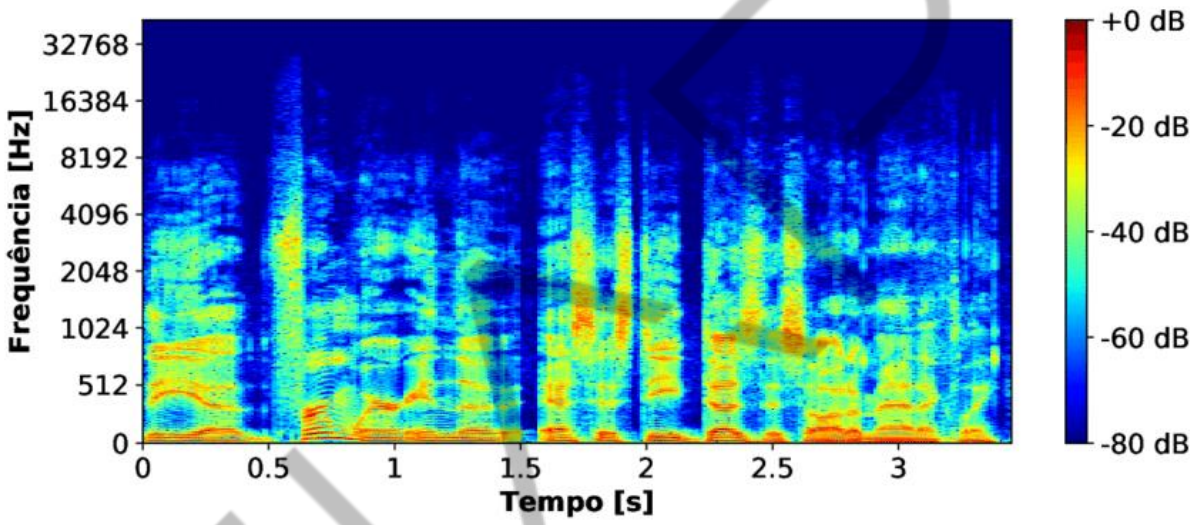

Figura 10 - Exemplo de um espectrograma
Fonte: ResearchGate (2020)

Os espectrogramas são ferramentas poderosas para a análise detalhada de sinais de áudio, pois permitem visualizar padrões complexos de frequências que mudam ao longo do tempo.

Além disso, os espectrogramas são úteis para identificar anomalias ou problemas em gravações de áudio. Por exemplo, em um estúdio de gravação, um engenheiro de som pode utilizar um espectrograma para identificar e remover ruídos indesejados, como zumbidos ou interferências. Outro exemplo de aplicação é na área da biomedicina, onde espectrogramas podem ser utilizados para analisar sons cardíacos ou pulmonares, ajudando a detectar condições médicas anormais.

Assim, podemos afirmar que o espectro de áudio e os espectrogramas são ferramentas essenciais para a análise e processamento de som. O espectro permite visualizar as frequências presentes em um sinal de áudio e suas amplitudes, enquanto

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

os espectrogramas mostram como essas frequências mudam ao longo do tempo. Juntas, essas representações fornecem uma compreensão profunda da estrutura e dinâmica dos sinais de áudio, permitindo aplicações avançadas em diversas áreas.

2.4 Princípios da fala

A sintetização de fala, ou síntese de voz, é o processo de gerar discurso artificialmente, de modo que soem como a fala humana. Este campo da engenharia de fala tem sido estudado e desenvolvido por décadas, com várias técnicas tradicionais que não envolvem modelos avançados de Deep Learning. Estas técnicas tradicionais baseiam-se em princípios da acústica, linguística e processamento de sinais para criar voz sintetizada de maneira eficiente e compreensível.

A sintetização de fala começa com a conversão de texto em fala. Este processo envolve várias etapas que vão desde a análise e processamento do texto até a geração do som final. Os principais componentes estão ilustrados na figura “Aplicações de data augmentation no domínio do espectrograma”.

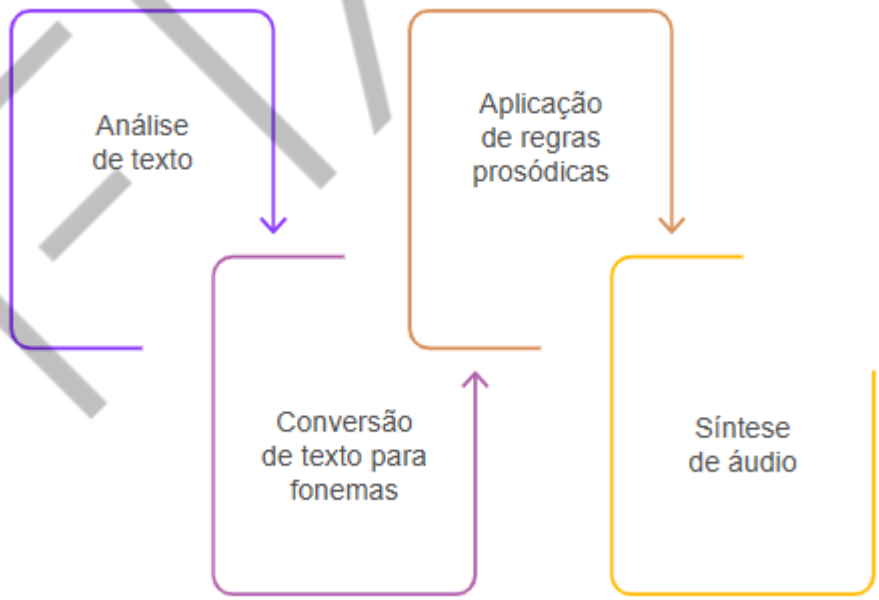

Figura 11 - Aplicações de data augmentation no domínio do espectrograma
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

1. **Análise de texto:** o texto é analisado para identificar palavras, frases, pontuações e outras características linguísticas. Esta análise

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

ajuda a determinar a **prosódia** — parte da linguística que estuda a entonação, o ritmo, o acento da linguagem falada e demais atributos correlatos na fala.

2. **Conversão de texto para fonemas:** o texto é convertido em uma sequência de fonemas, que são as unidades básicas de som em uma língua. Por exemplo, a palavra "fala" pode ser dividida em fonemas /f/, /a/, /l/, /a/. Veja outros exemplos na figura a seguir:

TÁXI	/t/ /a/ /k/ /s/ /i/	4 letras e 5 fonemas.
CAMPO	/k/ /ã/ /p/ /o/	5 letras e 4 fonemas.
HUMOR	/u/ /m/ /o/ /r/	5 letras e 4 fonemas.
VERMELHO	/v/ /e/ r/ /m/ /e/ /lh/ /o/	8 letras e 7 fonemas.
LUVA	/l/ /u/ /v/ /a/	4 letras e 4 fonemas.
CARRO	c/ /a/ /rr/ /o/	5 letras e 4 fonemas.
QUEIJO	/qu/ /e/ /i/ /j/ /o/	6 letras e 5 fonemas.
FIXO	/f/ /i/ /k/ /s/ /o/	4 letras e 5 fonemas.
BOLA	/b/ /o/ /l/ /a/	4 letras e 4 fonemas.
CHUVA	/x/ /u/ /v/ /a/	5 letras e 4 fonemas.
HUMANO	/u/ /m/ /a/ /n/ /o/	6 letras e 5 fonemas.
DOZE	/d/ /o/ /z/ /e/	4 letras e 4 fonemas.
BONÉ	/b/ /o/ /n/ /é/	4 letras e 4 fonemas.
PÁSSARO	/p/ /á/ /ss/ /a/ /r/ /o/	7 letras e 6 fonemas.

Figura 12 - Exemplos de fonemas
Fonte: Toda Matéria (2025)

3. **Aplicação de regras prosódicas:** as regras prosódicas são aplicadas para ajustar a entonação, o ritmo e a ênfase das palavras na frase, tornando a fala mais natural e expressiva.

4. **Síntese de áudio:** a sequência de fonemas é convertida em ondas sonoras através de técnicas de síntese de áudio, gerando a fala final.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.5 Etapas do reconhecimento da fala

O processo de reconhecimento de fala envolve várias etapas complexas que transformam o som capturado em texto compreensível e acionável. Cada etapa desempenha um papel crucial na precisão e eficácia do sistema, e existem diversas ferramentas e tecnologias disponíveis para suportar cada uma dessas etapas. A seguir, detalhamos as principais etapas e as ferramentas associadas ao reconhecimento de fala. Observe a figura:

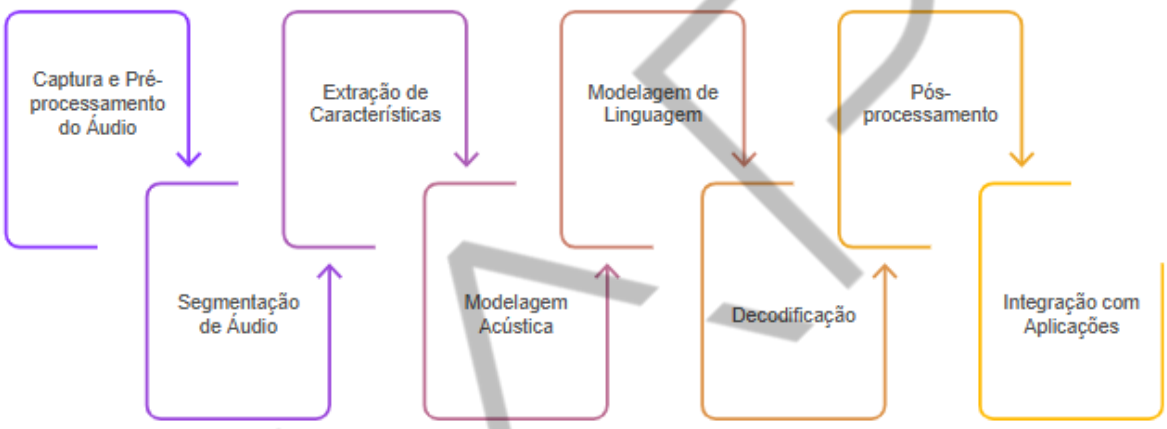

Figura 13 - Etapas do reconhecimento da voz
Fonte: Elaborado pelo autor (2025)

Detalhando cada uma das etapas, temos:

- **Captura e pré-processamento do áudio:** a primeira etapa no reconhecimento de fala é a captura do áudio, onde a fala é gravada por meio de microfones ou outros dispositivos de entrada de som. Durante essa fase, é importante garantir que o áudio capturado seja de alta qualidade, minimizando ruídos e interferências.
- **Segmentação do áudio:** depois de capturado, o áudio é segmentado em pequenas partes, chamadas frames, que geralmente duram entre 10 e 25 milissegundos. Cada frame contém uma pequena parte do sinal de áudio, permitindo que o sistema analise a fala em pequenos pedaços, o que é essencial para a precisão do reconhecimento.
- **Extração de características:** nesta etapa, o áudio segmentado é analisado para extrair características ou features que representem as informações

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

essenciais da fala. Essas características são cruciais para que o sistema de reconhecimento possa identificar padrões e correlacioná-los com palavras e frases.

- **Modelagem acústica:** a modelagem acústica envolve a criação de modelos que representam a relação entre as características extraídas e as unidades sonoras da fala (como fonemas). Estes modelos são utilizados para identificar quais palavras ou frases estão sendo ditas com base nos padrões sonoros.
- **Modelagem de linguagem:** enquanto a modelagem acústica foca nos sons, a modelagem de linguagem foca em como as palavras são combinadas em uma frase. Esta etapa envolve o uso de modelos probabilísticos para prever a sequência mais provável de palavras com base nas características extraídas e no contexto da fala.
- **Decodificação:** nesta fase, as informações das etapas de modelagem acústica e de linguagem são combinadas para gerar a sequência de palavras mais provável que corresponde ao áudio original. Esta é a etapa final que transforma o som em texto legível.
- **Pós-processamento:** após a decodificação, o texto gerado pode ser processado adicionalmente para melhorar a legibilidade e a precisão. Isso pode incluir correções gramaticais, normalização de texto, ou até mesmo a aplicação de regras de negócios específicas.
- **Integração com aplicações:** finalmente, o texto processado pode ser integrado em diversas aplicações, como assistentes virtuais, sistemas de atendimento ao cliente, ou transcrição automática. Esta etapa envolve a conexão dos resultados do reconhecimento de fala com outras partes de um sistema mais amplo, permitindo a automação de processos e a criação de interfaces de usuário baseadas em voz.

Embora o processo de reconhecimento de fala envolva várias etapas complexas, existem soluções e bibliotecas prontas que abstraem todos esses passos, facilitando a implementação dessa tecnologia em diversas aplicações. Ferramentas

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

como **Google Speech-to-Text, IBM Watson Speech to Text**, **Microsoft Azure Speech**, e bibliotecas open-source como **SpeechRecognition** e **DeepSpeech** oferecem APIs e interfaces de fácil integração que automatizam o processo de captura, pré-processamento, modelagem e decodificação de fala, permitindo que desenvolvedores e empresas implementem reconhecimento de voz de alta qualidade sem precisar lidar diretamente com cada uma dessas etapas técnicas.

Isso torna a tecnologia acessível e aplicável em uma ampla gama de setores, desde assistentes virtuais até transcrições automatizadas, democratizando o uso do reconhecimento de fala. Com a contínua evolução dessas tecnologias, o reconhecimento de fala continuará a desempenhar um papel central em como interagimos com dispositivos e sistemas no dia a dia.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 MODELOS DE DEEP LEARNING PARA ÁUDIO

3.1 Features tradicionais e modernas

O processamento de áudio, assim como qualquer outro dado complexo, também já era explorado antes do advento de Aprendizado Profundo. A diferença mais marcante estava na maneira como a Inteligência Artificial clássica gerava e representava features a partir de dados complexos. Vejamos o comparativo do quadro “Comparação de features de dados complexos”, que descreve o tipo de dado complexo com sua aplicação e as features pré e pós Deep Learning.

Aplicação	Features clássicas	Features em Deep Learning
Computer Vision (imagens e vídeos).	Features feitas à mão usando algoritmos para detectar cantos, arestas, pontos de interesse, faces etc.	Features diretamente dos pixels.
Processamento de Linguagem Natural (textos).	N-gramas e frequência de termos, vocabulários	Word embeddings.
Processamento de Linguagem Natural (áudios).	Usar fonética para extrair fonemas. Isso exige conhecimento específico de domínio para ajustar o sistema para um melhor desempenho.	Imagens de espectrogramas geradas a partir dos áudios.

Quadro 1 - Comparação de features de dados complexos
Fonte: Elaborado pelo autor (2025)

Assim, se torna evidente que a geração dos espectrogramas é uma das etapas fundamentais para trabalhar com Aprendizado de Máquina a partir de áudios. Isso facilita o processo, pois a partir do momento que geramos o espectrograma, estamos diante de uma imagem. Portanto, podemos voltar às técnicas clássicas de Redes Neurais Convolucionais (CNNs) para extração de features.

3.2 Aumentação de dados

O Data Augmentation é uma técnica essencial no treinamento de modelos de Deep Learning, especialmente quando se lida com conjuntos de dados limitados. No domínio do áudio, uma abordagem eficaz é transformar os sinais de áudio em imagens

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de espectrogramas e aplicar técnicas de augmentação, tanto no domínio do áudio quanto no domínio das imagens.

3.2.1 Técnicas de Data Augmentation no domínio do áudio

Nesta etapa, vamos aplicar transformações nos áudios brutos antes de gerar espectrogramas. Cada áudio modificado irá gerar espectrogramas diferentes:

- **Adição de ruído:** consiste em adicionar ruído branco ou ruído de fundo ao sinal de áudio original. Isso aumenta a robustez do modelo a ruídos ambientes e melhora a generalização.
- **Deslocamento temporal:** consiste em deslocar o sinal de áudio para frente ou para trás no tempo. Isso simula variações temporais que podem ocorrer na captura do áudio, aumentando a variabilidade dos dados.
- **Alteração de velocidade/pitch:** trata-se de alterar a velocidade de reprodução do áudio, o que também afeta o pitch. Esta perturbação introduz variações na frequência e no tempo, simulando diferentes condições de gravação.
- **Equalização aleatória:** trata-se de aplicar filtros de equalização aleatórios ao sinal de áudio, simulando diferentes ambientes acústicos e equipamentos de gravação.

A próxima figura ilustra os efeitos de aplicação de tais técnicas nos áudios. Em magenta temos o áudio original e em verde o resultado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

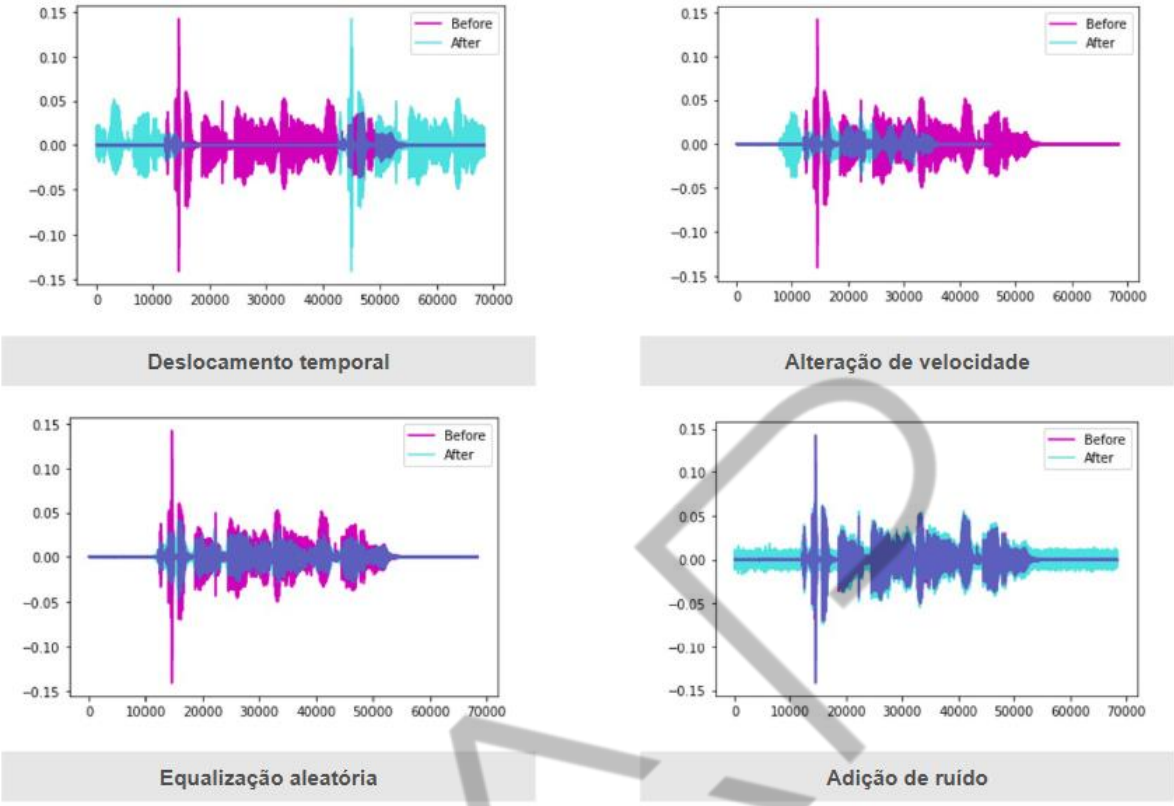

Figura 14 - Aplicações de Data Augmentation no domínio do áudio
Fonte: Towards Data Science (2021)

3.2.2 Técnicas de Data Augmentation no domínio do espectrograma

Aqui estamos aplicando transformações nos espectrogramas, gerando variações sem alterar o áudio original. Porém, é preciso tomar cuidado nessa etapa! As augmentações típicas de imagem não se aplicam aqui, pois podem modificar drasticamente o som representado pelo espectrograma, introduzindo vieses nos modelos de Aprendizado de Máquina.

O objetivo de alterar os espectrogramas é forçar o modelo a aprender representações mais robustas, não dependentes de partes específicas daquela imagem (ilustrado na figura “Aplicações de Data Augmentation no domínio do espectrograma”). Podemos fazer:

- **Máscara de frequência:** mascarar aleatoriamente faixa de frequências consecutivas, por meio da adição de barras horizontais no espectrograma.
- **Máscara de tempo:** bloquear aleatoriamente faixas de tempo do espectrograma, por meio da adição de barras verticais no espectrograma.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

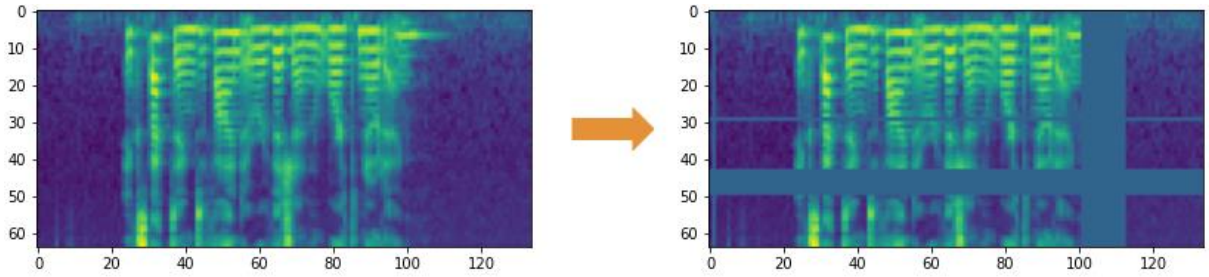

Figura 15 - Aplicações de Data Augmentation no domínio do espectrograma
Fonte: Towards Data Science (2021)

3.3 Modelos profundos mais utilizados

Nos últimos anos, o Deep Learning revolucionou o campo da sintetização de fala, permitindo a geração de fala sintetizada com uma naturalidade e fluidez que superam as técnicas tradicionais. Os avanços em Redes Neurais Profundas possibilitaram modelos mais complexos e eficazes, que capturam as nuances da prosódia e da articulação humana de maneira mais sofisticada.

A sintetização de fala com Deep Learning geralmente envolve a transformação de texto em fala por meio de Redes Neurais Profundas que aprendem a mapear diretamente o texto para ondas sonoras. Esse processo pode ser dividido em duas etapas principais:

- **Conversão Texto-Para-Espectrograma (*Text-to-Spectrogram*):** o texto é convertido em um espectrograma, que é uma representação visual da frequência do som ao longo do tempo. Modelos como Tacotron e Tacotron 2 são exemplos notáveis dessa abordagem. Esses modelos utilizam RNNs e CNNs para gerar espectrogramas a partir do texto.
- **Conversão Espectrograma-Para-Onda Sonora (*Spectrogram-to-Waveform*):** o espectrograma gerado é convertido em uma onda sonora (*audio Waveform*). Modelos como WaveNet, desenvolvido pelo Google DeepMind, são amplamente utilizados nessa etapa. WaveNet é uma Rede Neural Generativa que modela a distribuição de probabilidade das ondas sonoras, gerando áudio de alta qualidade a partir dos espectrogramas.

WaveNet é um modelo generativo desenvolvido pela DeepMind, que revolucionou a sintetização de fala ao oferecer uma qualidade de áudio sem

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

precedentes. A principal inovação do WaveNet é sua capacidade de modelar diretamente a distribuição de probabilidade conjunta das ondas sonoras, gerando áudio uma amostra por vez.

Além disso, a WaveNet é flexível, podendo ser condicionada em diferentes tipos de dados, como texto ou espectrogramas, permitindo aplicações diversas como TTS e geração de música. Por outro lado, o processo de geração é computacionalmente intensivo, pois gera uma amostra de cada vez, exigindo poder de processamento significativo.

Por sua vez, o reconhecimento de fala com Deep Learning também evoluiu rapidamente com o uso de arquiteturas como DeepSpeech, baseada em redes recorrentes, e Wav2Vec 2.0, da Meta AI, que usa transformadores para aprender representações diretamente de ondas sonoras. Para quem deseja aplicar essas tecnologias de forma prática, bibliotecas como SpeechRecognition, DeepSpeech, ESPnet, torchaudio, Transformers (Hugging Face) e OpenAI Whisper oferecem interfaces acessíveis e modelos pré-treinados que permitem realizar reconhecimento de fala com poucas linhas de código, tornando essa tecnologia cada vez mais disponível para aplicações reais.

Além disso, treinar tais redes requer grandes quantidades de dados e recursos computacionais, o que pode ser uma barreira para diversas aplicações. Portanto, exploraremos a sintetização e reconhecimento de voz por meio de bibliotecas em Python baseadas em modelos profundos já treinados.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 HANDS ON: PRATICANDO STT E TTS

Agora que conhecemos os aspectos centrais do processamento de áudio, está na hora de praticarmos algumas implementações pertinentes. Nós faremos três códigos diferentes: um para geração de espectrogramas, outro para sintetização de fala e o terceiro para reconhecimento de voz.

Nossas práticas serão realizadas em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

4.1 Gerando espectrogramas

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 5a), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Dica: para o código deste capítulo, utilize os arquivos de apoio "**audio_file.wav**".

Sobre a base de dados:

Este é um exemplo de arquivo de áudio livre. Não há voz ou narração, apenas uma música curta. Nosso objetivo é extrair o espectrograma deste arquivo de áudio.

Vamos para o código!

Começaremos com o import da biblioteca Librosa e carregar o arquivo:

```
import librosa

# Carregar o arquivo
AUDIO_FILE = "audio_file.wav"

samples, sample_rate = librosa.load(AUDIO_FILE, sr=None)
#SR: sample rate
```

Código-fonte 1 - Imports e carregamento do arquivo de áudio
Fonte: Elaborado pelo autor (2025)

Este primeiro código não gera saída. Vamos exibir o *sample rate* padrão:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
sample_rate
```

Código-fonte 2 - Exibindo o *sample rate* padrão da biblioteca Librosa
Fonte: Elaborado pelo autor (2024)

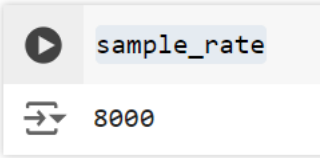

Figura 16 - Saída do Código-fonte 2
Fonte: Elaborado pelo autor (2025)

Podemos plotar o arquivo de áudio no domínio do tempo, ou seja, mostrar suas variações de amplitude ao longo do tempo:

```
# Plotar o áudio no aspectro do tempo
import matplotlib.pyplot as plt

plt.figure(figsize=(14,5))
librosa.display.waveshow(samples, sr=sample_rate)
```

Código-fonte 3 - Exibindo as amplitudes do áudio
Fonte: Elaborado pelo autor (2025)

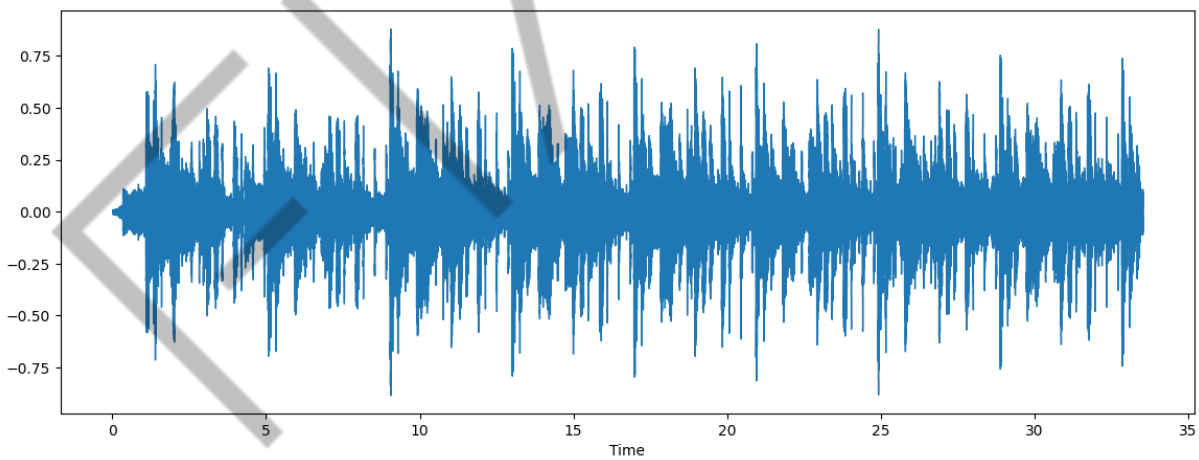

Figura 17 - Saída do Código-fonte 3
Fonte: Elaborado pelo autor (2025)

Também é possível plotar o próprio arquivo de áudio! Para reproduzi-lo, basta clicar no “play”, conforme a figura “Saída do Código-fonte 4”.

```
from IPython.display import Audio

Audio(AUDIO_FILE)
```

Código-fonte 4 - Exibindo o próprio arquivo de áudio
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

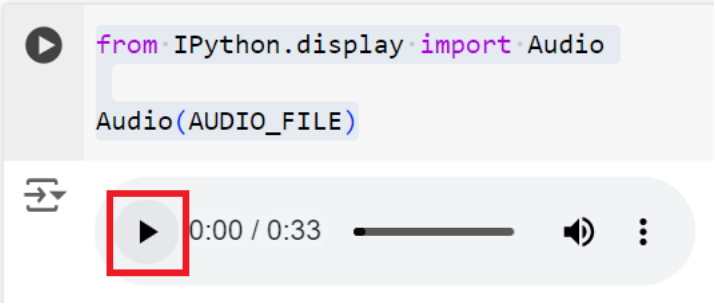

Figura 18 - Saída do Código-fonte 4
Fonte: Elaborado pelo autor (2025)

Agora vamos extrair o espectrograma diretamente do áudio via Librosa. Aqui aplicamos uma “Short-Time Fourier Transform” (Transformada de Fourier de Curto Prazo), que calcula a transformada de Fourier de curto prazo de uma série temporal, ou seja, ela transforma o sinal de áudio (no domínio do tempo) para o domínio da frequência:

```
# Extrair o espectrograma
sgram = librosa.stft(samples)
librosa.display.specshow(sgram)
```

Código-fonte 5 - Exibindo o espectrograma padrão da biblioteca Librosa
Fonte: Elaborado pelo autor (2025)

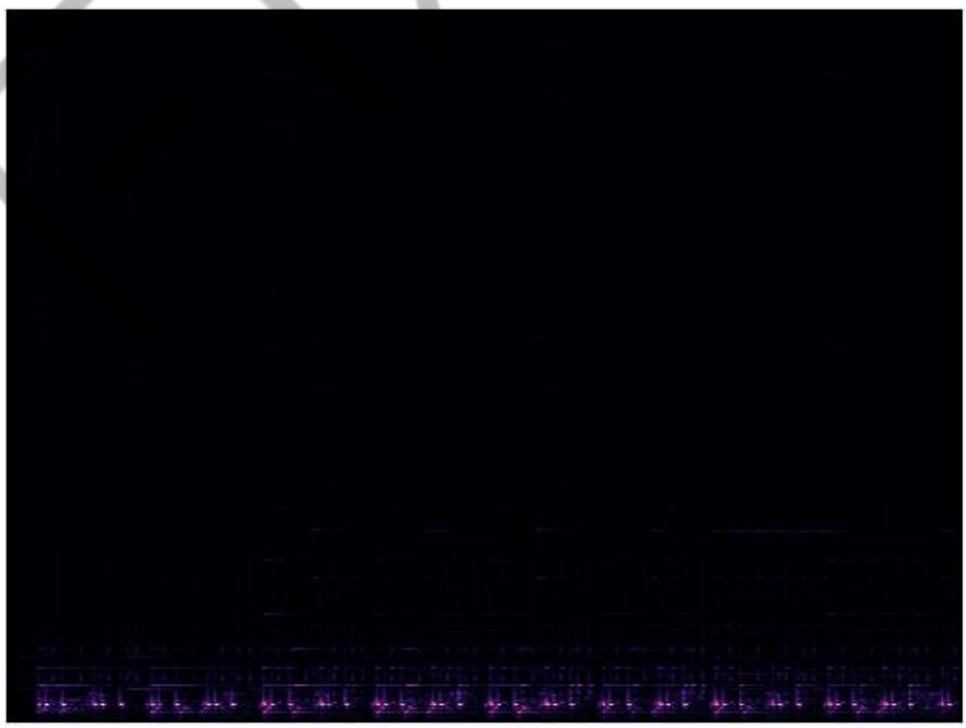

Figura 19 - Saída do Código-fonte 5
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observe que o espectrograma da figura “Saída do Código-fonte 5” não é muito informativo, pois a região diferente do fundo preto é muito pequena. Então vamos separar o espectrograma em sua magnitude e fase (esta é descartada “_”). Em seguida, a partir da magnitude, calculamos o espectrograma na escala de Mel e depois o plotamos. A figura “Saída do Código-fonte 6” já mostra uma melhoria no resultado:

```
     # Valorizar o espectro percebido pelo humano
     sgram_mag, _ = librosa.magphase(sgram)
     mel_scale_sgram                                        =
librosa.feature.melspectrogram(S=sgram_mag,
                                                        sr=samp
le_rate)
     librosa.display.specshow(mel_scale_sgram)
```

Código-fonte 6 - Exibindo a magnitude do espectrograma padrão da biblioteca Librosa
Fonte: Elaborado pelo autor (2025)

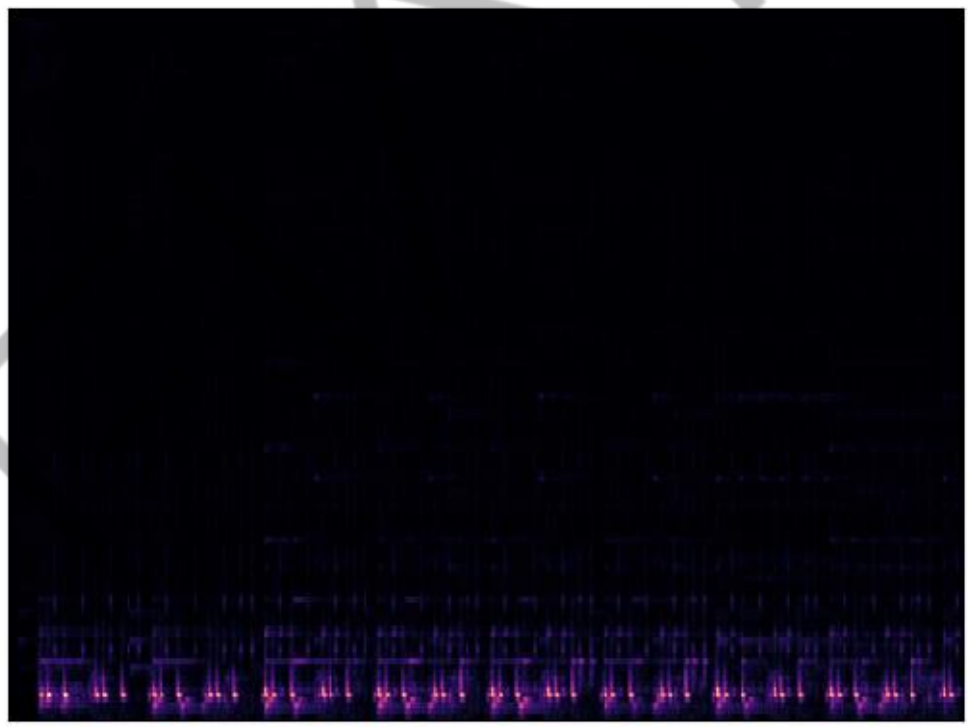

Figura 20 - Saída do Código-fonte 6
Fonte: Elaborado pelo autor (2025)

Para tornar as nuances do áudio ainda mais perceptíveis, vamos levá-lo para a escala de decibéis:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Converter para escala de decibéis
import numpy as np

mel_sgram = librosa.amplitude_to_db(mel_scale_sgram,
ref=np.min)

librosa.display.specshow(mel_sgram, sr=sample_rate,
                         x_axis="time", y_axis="mel")
plt.colorbar(format="%+2.0f dB")
```

Código-fonte 7 - Convertendo o espectrograma padrão para a escala decibéis
Fonte: Elaborado pelo autor (2025)

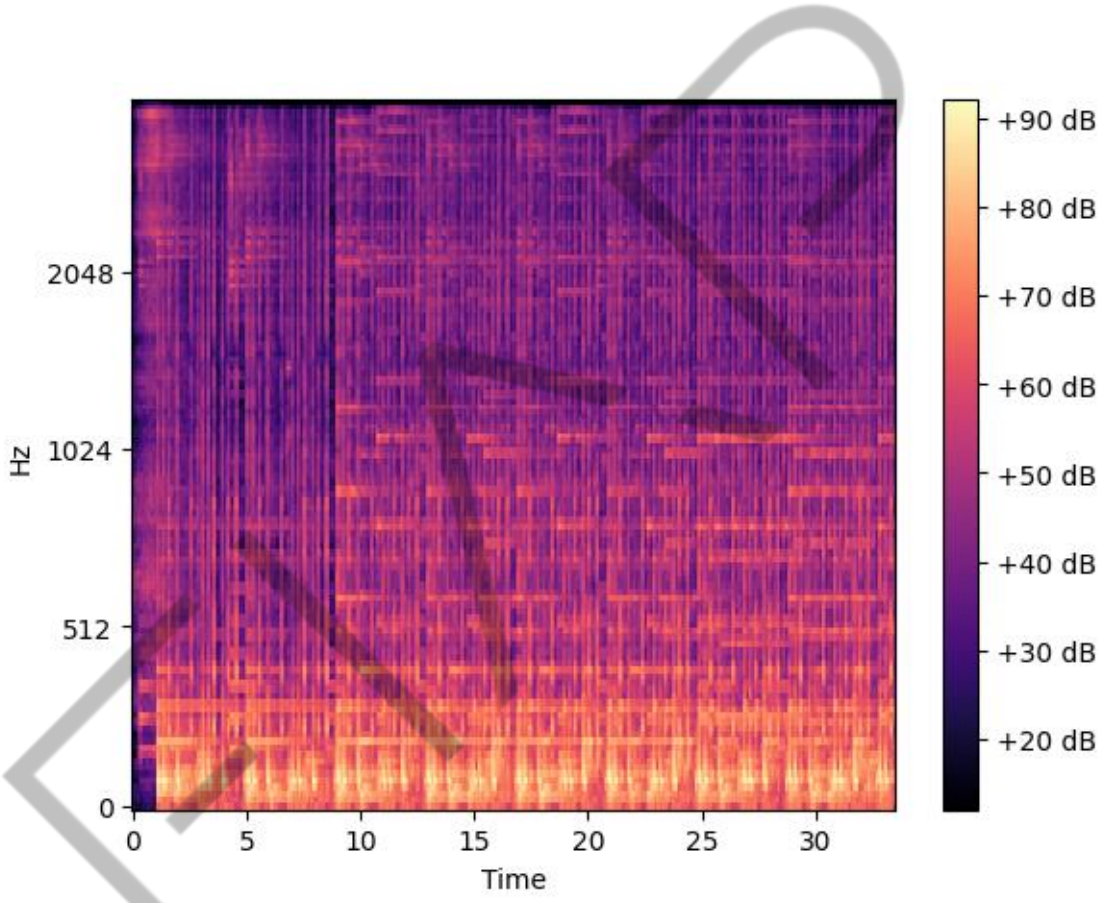

Figura 21 - Saída do Código-fonte 7
Fonte: Elaborado pelo autor (2025)

A figura “Saída do Código-fonte 7” exibe perfeitamente as variações do arquivo de áudio. Esta versão do espectrograma é muito mais relevante para extrair informações. A imagem ainda exibe o tempo do áudio em segundos e as variações de frequência em Hertz. Também temos a legenda, que é a escala em decibéis.

Para limpar a imagem e ficar apenas com as informações visuais, podemos plotá-la novamente por meio do seguinte comando:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Ploto a imagem no formato consumível para CNN
librosa.display.specshow(mel_sgram, sr=sample_rate);
```

Código-fonte 8 - Exibindo o espectrograma em escala decibéis para processamento de CNNs
Fonte: Elaborado pelo autor (2025)

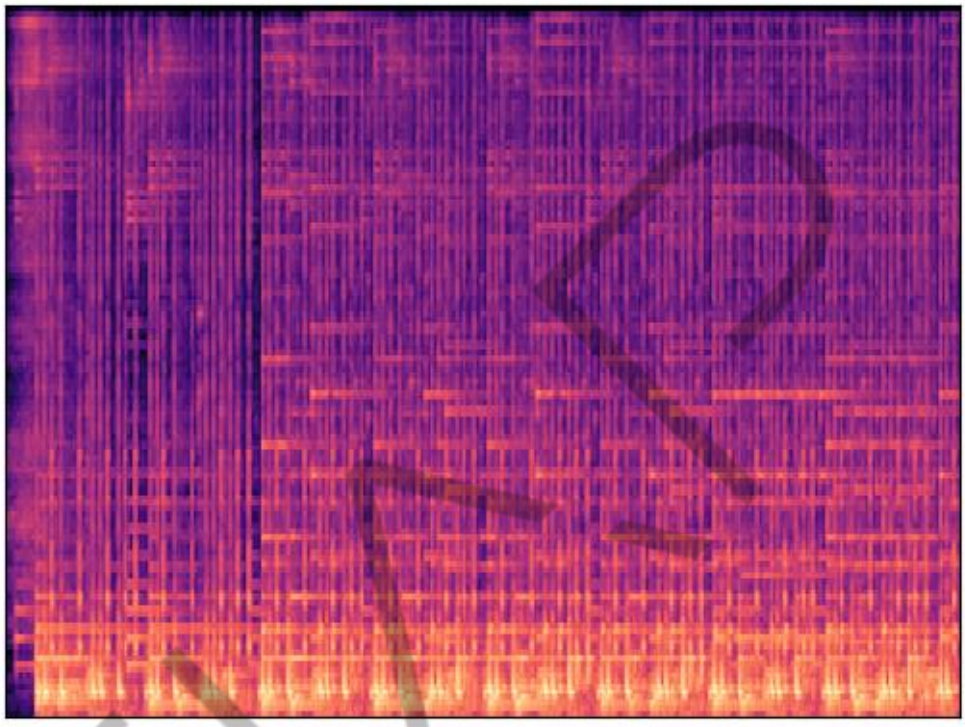

Figura 22 - Saída do Código-fonte 8
Fonte: Elaborado pelo autor (2025)

Pronto! A figura “Saída do Código-fonte 8” mostra o espectrograma na escala de decibéis, com toda a informação visual claramente exposta. A partir desta imagem podemos alimentar uma CNN para extrair informações numéricas para alimentar um modelo de Aprendizado de Máquina. Tudo dependerá de qual label escolheríamos para este áudio e qual tarefa de Aprendizado de Máquina queríamos construir: por exemplo, reconhecer o estilo musical a partir da música.

O processo de treinar um modelo a partir de espectrogramas exigiria extrair espectrogramas de cada arquivo de áudio a partir do processo que acabamos de ver!

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 TTS

Novamente no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 5b), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Primeiramente vamos conhecer a biblioteca gTTS, que é a solução do Google para Text-to-Speech. O código a seguir faz a instalação da biblioteca e mostra sua utilização padrão, gerando o áudio a partir de um texto:

```
# Instalação e utilização padrão da biblioteca
!pip install gtts
import gtts as g

voz = g.gTTS("Olá, pessoal!", lang="pt")
voz.save("voz.mp3")

from IPython.display import Audio
Audio("voz.mp3")
```

Código-fonte 9 - Utilização padrão do gTTS
Fonte: Elaborado pelo autor (2025)

Note que a utilização é muito simples! Basta indicar a frase a ser convertida em áudio e em qual língua ela está escrita (o que implica a língua do áudio também). Isso retorna um arquivo de áudio que pode ser salvo como ".mp3" e posteriormente carregado no próprio notebook. A saída está na próxima figura:

```
Requirement already satisfied: gtts in /usr/local/lib/python3.10/dist-packages (2.5.2)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.10/dist-packages (from gtts) (2.31.0)
Requirement already satisfied: click<8.2,>=7.1 in /usr/local/lib/python3.10/dist-packages (from gtts) (8.1.7)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.27->gtts) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.27->gtts) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.27->gtts) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.27->gtts) (2024.7.4)
```

0:02 / 0:02

Figura 23 - Saída do Código-fonte 9
Fonte: Elaborado pelo autor (2025)

Ao clicar no "play" da figura "Saída do Código-fonte 9" é possível ouvir a reprodução do áudio gerado a partir da frase de entrada.

Você pode estar se pensando: "A gTTS possui suporte para outras línguas? Quais?". Vamos descobrir com o código a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Exibindo a lista de línguas
g.lang.tts_langs()
```

Código-fonte 10 - Línguas suportadas pelo gTTS
Fonte: Elaborado pelo autor (2025)

A figura a seguir mostra a saída parcial (a lista é bem maior!). Ela indica a língua e o código para uso na biblioteca — a palavra com duas letras, por exemplo, “de” para Alemão (German):

```
{'af': 'Afrikaans',
 'am': 'Amharic',
 'ar': 'Arabic',
 'bg': 'Bulgarian',
 'bn': 'Bengali',
 'bs': 'Bosnian',
 'ca': 'Catalan',
 'cs': 'Czech',
 'cy': 'Welsh',
 'da': 'Danish',
 'de': 'German',
 'el': 'Greek',
 'en': 'English',
 'es': 'Spanish',
 'et': 'Estonian',
 'eu': 'Basque',
 'fi': 'Finnish',
 'fr': 'French',
```

Figura 24 - Saída do Código-fonte 10
Fonte: Elaborado pelo autor (2025)

Dica: para o próximo código, utilize o arquivo de apoio “**frases.txt**”.

Podemos ler um arquivo de texto no qual cada linha contém o acrônimo da língua utilizada e na sequência uma frase nesta língua. Ambos os campos estão separados por “;”. O trecho do código-fonte “Gerando áudios diversos no gTTS” lê o arquivo e exibe cada frase no idioma apontado.

```
arq = open("frases.txt", "r")
for linha in arq.readlines():
  lang, texto = linha.split(";")
  voz = g.gTTS(texto, lang=lang)
  voz.save("linha.mp3")
  display( Audio("linha.mp3") )
```

Código-fonte 11 - Gerando áudios diversos no gTTS
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Reproduza cada um dos áudios clicando nos “plays” da figura “Saída do Código-fonte 11”. A mesma frase é dita em quatro idiomas distintos (Português, Inglês, Francês e Italiano).

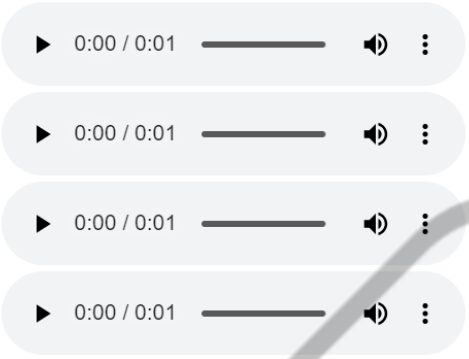

Figura 25 - Saída do Código-fonte 11
Fonte: Elaborado pelo autor (2025)

4.3 SST

No Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 5c), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Começaremos com a instalação da biblioteca SpeechRecognition e suas dependências:

```
!pip install ffmpeg-python
!pip install SpeechRecognition
!apt install libasound2-dev portaudio19-dev libportaudio2
libportaudiocpp0 ffmpeg
```

Código-fonte 12 - Instalação das bibliotecas
Fonte: Elaborado pelo autor (2025)

A seguir está a saída confirmando a instalação:

```
Requirement already satisfied: ffmpeg-python in /usr/local/lib/python3.10/dist-packages (0.2.0)
Requirement already satisfied: future in /usr/local/lib/python3.10/dist-packages (from ffmpeg-python) (1.0.0)
Requirement already satisfied: SpeechRecognition in /usr/local/lib/python3.10/dist-packages (3.10.4)
Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.10/dist-packages (from SpeechRecognition) (2.32.3)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (from SpeechRecognition) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests>=2.26.0->SpeechRecognition) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests>=2.26.0->SpeechRecognition) (3.8)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests>=2.26.0->SpeechRecognition) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests>=2.26.0->SpeechRecognition) (2024.7.4)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libasound2-dev is already the newest version (1.2.6.1-1ubuntu1).
libportaudio2 is already the newest version (19.6.0-1.1).
libportaudiocpp0 is already the newest version (19.6.0-1.1).
portaudio19-dev is already the newest version (19.6.0-1.1).
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 49 not upgraded.
```

Figura 26 - Saída do Código-fonte 12
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como estamos executando o código no Colab, dependemos do navegador Web para ter acesso aos recursos do computador, como o microfone. O próximo código solicita que o navegador peça ao usuário a liberação de acesso ao microfone, para captura da voz. Ele não possui saída:

```
# Códigos para solicitar permissão de uso do microfone do computador pelo browser
"""
Some references:
https://blog.addpipe.com/recording-audio-in-the-browser-using-pure-html5-and-minimal-javascript/
https://stackoverflow.com/a/18650249
https://hacks.mozilla.org/2014/06/easy-audio-capture-with-the-mediarecorder-api/
https://air.ghost.io/recording-to-an-audio-file-using-html5-and-js/
https://stackoverflow.com/a/49019356
"""
from IPython.display import HTML, Audio
from google.colab.output import eval_js
from base64 import b64decode
import numpy as np
from scipy.io.wavfile import read as wav_read
import io
import ffmpeg

AUDIO_HTML = """
<script>
var my_div = document.createElement("DIV");
var my_p = document.createElement("P");
var my_btn = document.createElement("BUTTON");
var t = document.createTextNode("Press to start recording");

my_btn.appendChild(t);
//my_p.appendChild(my_btn);
my_div.appendChild(my_btn);
document.body.appendChild(my_div);

var base64data = 0;
var reader;
var recorder, gumStream;
var recordButton = my_btn;

var handleSuccess = function(stream) {
  gumStream = stream;
  var options = {
     //bitsPerSecond: 8000, //chrome seems to ignore, always 48k
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
          mimeType : 'audio/webm;codecs=opus'
          //mimeType : 'audio/webm;codecs=pcm'
        };
        //recorder = new MediaRecorder(stream, options);
        recorder = new MediaRecorder(stream);
        recorder.ondataavailable = function(e) {
          var url = URL.createObjectURL(e.data);
          var preview = document.createElement('audio');
          preview.controls = true;
          preview.src = url;
          document.body.appendChild(preview);

          reader = new FileReader();
          reader.readAsDataURL(e.data);
          reader.onloadend = function() {
            base64data = reader.result;
            //console.log("Inside FileReader:" + base64data);
          }
        };
        recorder.start();
        };

      recordButton.innerText = "Recording... press to stop";

      navigator.mediaDevices.getUserMedia({audio:
true}).then(handleSuccess);

      function toggleRecording() {
        if (recorder && recorder.state == "recording") {
            recorder.stop();
            gumStream.getAudioTracks()[0].stop();
             recordButton.innerText = "Saving the recording...
pls wait!"
        }
      }

      // https://stackoverflow.com/a/951057
      function sleep(ms) {
        return new Promise(resolve => setTimeout(resolve, ms));
      }

      var data = new Promise(resolve=>{
      //recordButton.addEventListener("click",
toggleRecording);
      recordButton.onclick = ()=>{
      toggleRecording()

      sleep(2000).then(() => {
        // wait 2000ms for the data to be available...
        // ideally this should use something like await...
        //console.log("Inside data:" + base64data)
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        resolve(base64data.toString())

      });

      }
      });

      </script>
      """

      def get_audio():
        display(HTML(AUDIO_HTML))
        data = eval_js("data")
        binary = b64decode(data.split(',')[1])

        process = (ffmpeg
          .input('pipe:0')
          .output('pipe:1', format='wav')
               .run_async(pipe_stdin=True,    pipe_stdout=True,
pipe_stderr=True, quiet=True, overwrite_output=True)
        )
        output, err = process.communicate(input=binary)

        riff_chunk_size = len(output) - 8
        # Break up the chunk size into four bytes, held in b.
        q = riff_chunk_size
        b = []
        for i in range(4):
            q, r = divmod(q, 256)
            b.append(r)

        # Replace bytes 4:8 in proc.stdout with the actual size
of the RIFF chunk.
        riff = output[:4] + bytes(b) + output[8:]

        sr, audio = wav_read(io.BytesIO(riff))

        return audio, sr
```

Código-fonte 13 - Solicitando a captura de áudio pelo microfone
Fonte: Elaborado pelo autor (2025)

Agora, podemos tentar o próximo código. Ele irá gravar enquanto não clicarmos no botão “Stop”. Após a captura do áudio, ele salvará o arquivo no ambiente do Colab:

```
      # Obtém áudio do microfone via navegador
      audio, sr = get_audio()
```

Código-fonte 14 - Testando a captura do áudio
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Seguimos com os imports necessários. Esta célula não possui saída:

```
# Imports
import numpy as np
from scipy.io.wavfile import write
import speech_recognition as speechRecog
```

Código-fonte 15 - Realizando os imports necessários
Fonte: Elaborado pelo autor (2025)

Iremos definir a função que captura os bits e bytes capturados pelo microfone e salvar um arquivo ".wav" com o áudio. Esta célula não possui saída:

```
# Normalizar o retorno do microfone e salvar num ".wav"
def mic2wav(audio, wav_file):
  scaled = np.int16( audio/np.max( np.abs(audio) ) * 32767
)
  write(wav_file, 44100, scaled)
```

Código-fonte 16 - Salva num .wav o áudio capturado
Fonte: Elaborado pelo autor (2025)

A próxima função é responsável por transcrever o áudio em si baseado na biblioteca SpeechRecognition. Esta célula não possui saída:

```
# Transcreve o áudio
def transcreve_audio(wav_file):

  audioFile = speechRecog.AudioFile(wav_file)
  recon = speechRecog.Recognizer()

  with audioFile as source:
    audioData = recon.record(audioFile)

      texto      =      recon.recognize_google(audioData,
language="pt")

  return texto
```

Código-fonte 17 - Transcrição de áudio em português
Fonte: Elaborado pelo autor (2025)

Podemos ver um exemplo de uso da função criada:

```
transcreve_audio("teste.wav")
```

Código-fonte 18 - Exemplo de transcrição de áudio em português
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
transcreve_audio("teste.wav")
```

```
'neste momento ele Está capturando a minha voz vou parar nesse segundo'
```

Figura 27 - Saída do Código-fonte 18
Fonte: Elaborado pelo autor (2025)

Vamos para um exemplo mais sofisticado! O código-fonte “Exemplo de transcrição em loop de áudios em português” é um loop infinito que captura a voz do usuário e processa imprimindo o texto reconhecido. Caso o usuário diga a palavra “pare”, o loop é interrompido.

```
# Encadeando o reconhecimento da fala até comando "pare"
texto = ""
while texto != "pare":
  audio, _ = get_audio()
  mic2wav(audio, "som.wav")
  texto = transcreve_audio("som.wav")
  print("Você disse: ", texto)
```

Código-fonte 19 - Exemplo de transcrição em loop de áudios em português
Fonte: Elaborado pelo autor (2025)

```
Você disse:  estou gravando a primeira frase
Você disse:  teste número 2
Você disse:  computação é muito legal
Você disse:  trabalhar com áudio é sensacional
Você disse:  pare
Recording... press to stop
Recording... press to stop
Recording... press to stop
Recording... press to stop
Recording... press to stop
```

Figura 28 - Saída do Código-fonte 19
Fonte: Elaborado pelo autor (2025)

Claramente, a saída da figura “Saída do Código-fonte 19” depende da frase que você está falando. A cada vez que o botão “stop” é acionado, há o processamento daquele trecho e impressão do texto reconhecido.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A conclusão do nosso estudo sobre reconhecimento de fala nos leva a uma reflexão sobre as tecnologias de TTS e STT, que, embora diferentes em suas funções, se complementam de forma poderosa em muitas aplicações modernas.

O Speech-to-Text, como vimos, é a tecnologia que converte a fala em texto. Ela permite que dispositivos e sistemas entendam comandos de voz, transcrevam diálogos e façam buscas por meio de linguagem natural. O STT é fundamental para assistentes virtuais, transcrições automáticas, e diversas outras aplicações que exigem a interpretação de fala humana em tempo real ou a partir de gravações.

Por outro lado, o Text-to-Speech realiza o processo inverso: converte texto em fala. Essa tecnologia é crucial para tornar as interações com máquinas mais naturais e acessíveis, permitindo que dispositivos “falem” com os usuários, leiam textos em voz alta, e ofereçam feedback auditivo em uma variedade de contextos, desde assistentes pessoais até soluções de acessibilidade para pessoas com deficiência visual.

Quando combinados, TTS e STT formam a base para uma comunicação fluida e interativa entre humanos e máquinas. Por exemplo, em um assistente virtual, o STT permite que o sistema entenda o que o usuário está dizendo, enquanto o TTS possibilita que o assistente responda de forma audível. Esse ciclo de comunicação bidirecional torna a interação com a tecnologia mais intuitiva e eficiente, aproximando-a das interações humanas naturais.

Essa complementaridade é fundamental para o avanço das interfaces de voz e para a criação de experiências de usuário cada vez mais integradas e inclusivas, com um impacto significativo em áreas como acessibilidade, educação, saúde, e serviços ao consumidor. Felizmente, não precisamos treinar Redes Neurais muito complexas para lidar com TTS ou STT, pois existem boas bibliotecas e serviços em nuvem que implementam estes serviços.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

A particle executing linear shm. **Vedantu**, 2025. Disponível em: <https://www.vedantu.com/question-answer/a-particle-executing-linear-shm-its-time-period-class-11-physics-cbse-5f973e0ef231cf558a37382e>. Acesso em: 28 mar. 2025.

ABDEL-HAMID, Ossama, *et al.* **Convolutional neural networks for speech recognition**. IEEE/ACM Transactions on audio, speech, and language processing, 2014, 22.10: 1533-1545.

CAMPOS, António Miguel de. Ficheiro: Espectro harmônico. **Wikipedia**, 9 de maio de 2006. Disponível em: <https://pt.m.wikipedia.org/wiki/Ficheiro:Espectro_harm%C3%B3nico.jpg>. Acesso em: 28 mar. 2025.

COMO integrar o home theater com assistentes virtuais? **Audio Prime**, 11 de julho de 2023. Disponível em: <https://www.audioprime.com.br/blog/como-integrar-o-home-theater-com-assistentes-virtuais>. Acesso em: 28 mar. 2025.

CONHEÇA as tecnologias usadas para reconhecimento de voz. **Inforchannel**, 17 de junho de 2023. Disponível em: <https://inforchannel.com.br/2023/06/17/conheca-as-tecnologias-usadas-para-reconhecimento-de-voz/>. Acesso em: 28 mar. 2025.

DOSHI, Ketan. Audio Deep Learning Made Simple (Part 3): Data Preparation and Augmentation. **Towards Data Science**, 24 de fevereiro de 2021. Disponível em: <https://towardsdatascience.com/audio-deep-learning-made-simple-part-3-data-preparation-and-augmentation-24c6e1f6b52/>. Acesso em: 28 mar. 2025.

FERNANDES, Márcia. O que são fonemas (com exemplos). **Toda Matéria**, 2025. Disponível em: <https://www.todamateria.com.br/o-que-sao-fonemas/>. Acesso em: 28 mar. 2025.

GOUVÊA, Thiago Azeredo. Origem do universo e gravidade. **O Globo**, 2015. Disponível em: <http://educacao.globo.com/artigo/origem-do-universo-e-gravidade.html>. Acesso em: 28 mar. 2025.

IAZZETTA, Fernando. Comprimento de onda (lambida). **USP**, 2025. Disponível em: <https://iazzetta.eca.usp.br/tutor/acustica/comprimento/comprim.html>. Acesso em: 28 mar. 2025.

LANGUAGE Models Are Changing Everything. **Parse.ly**, 2025. Disponível em: <https://www.parse.ly/language-models-are-changing-everything/>. Acesso em: 28 mar. 2025.

MEHRISH, Ambuj, *et al.* A review of deep learning techniques for speech processing. **Information Fusion**, 2023, 99: 101869.

OLIVEIRA, L. M. Frequência do som. **UFRGS**, março de 2002. Disponível em: <https://www.if.ufrgs.br/cref/ntef/som/freq.html>. Acesso em: 28 mar. 2025.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

RECONHECIMENTO de fala. **CPQD**, 2017. Disponível em: <https://speechweb.cpqd.com.br/asr/docs/2.9/get_started/asr_intro.html>. Acesso em: 28 mar. 2025.

SAK, Haşim; SENIOR, Andrew; BEAUFAYS, Françoise. **Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition**. arXiv preprint arXiv:1402.1128, 2014.

SPADINI, Tito. Exemplo de mel-espectrograma de um sinal de voz masculina lendo a frase "The firmware in the SSD does this automatically". **ResearchGate**, fevereiro de 2020. Disponível em: <https://www.researchgate.net/figure/Figura-223-Exemplo-de-mel-espectrograma-de-um-sinal-de-voz-masculina-lendo-a-frase-The_fig9_339434039>. Acesso em: 28 mar. 2025.

VAN DEN OORD, Aaron, *et al*. **Wavenet:** A generative model for raw audio. arXiv preprint arXiv:1609.03499, 2016, 12.

WANG, Yuxuan, *et al.* **Tacotron:** A fully end-to-end text-to-speech synthesis model. arXiv preprint arXiv:1703.10135, 2017, 164.

ZAMAN, Khalid, *et al*. **A survey of audio classification using deep learning**. IEEE Access, 2023.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
CNNs	Redes Neurais Convolucionais.
DNNs	Deep Neural Networks — Redes Neurais Profundas.
GAN	Generative Adversarial Networks.
HMM	Hidden Markov Models.
LSTM	Long Short-Term Memory.
MFCC	Mel-Frequency Cepstral Coefficients.
PLP	Perceptual Linear Prediction.
RNNs	Redes Neurais Recorrentes.
STT	Speech-to-Text.
TTS	Text-to-Speech.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com